Swarmtest: Product Requirements Document

Working name. Hackathon edition v0.1.

Assumptions (change if wrong): team of 4, 24 build hours, local-only demo, access to Anthropic API credits. Adjust the plan in section 13 if any of these differ.

1. Summary

Swarmtest is a multi-agent testing system that puts a simulated community, made of normal users plus adaptive adversaries, inside a real running application and finds the failures that only appear when users interact: race conditions, abuse loopholes, moderation gaps, and economic exploits.

Existing AI testers send isolated personas through single flows. Swarmtest tests the app as a society, then proves what it found with replayable reproductions and a published detection rate.

One-liner: Existing AI testers check whether one user can finish a flow. We unleash a whole community, trolls and fraudsters included, on your app and find the failures that only appear when users collide.

Target customer: early-stage startups launching multi-user products where users affect each other (marketplaces first, then booking, fintech, and community apps). They find out how their app behaves when real users collide, before real users do.

Build order: Phase 1 is the core swarm, verifier, and benchmark (sections 1 to 15). Phase 2 adds memory through Meko (section 17) only after Phase 1 works; nothing in Phase 1 depends on Meko. Customers run Swarmtest as a CLI with a config file; a hosted API key is future work (section 16).

2. Problem

- Apps pass unit and end-to-end tests, then break when real users arrive. Many of the worst failures are interactions between users: overselling the last item, coupon stacking under concurrency, review rings, block bypasses, brigading.
- AI QA tools already exist and are good at single-user flows (for example crowd-test, Propolis, Faultline). In the ones we reviewed, personas run mostly in isolation, so inter-user failures are out of reach.
- Research simulators of online platforms (for example COSMOS and YSocial-style digital twins) model a platform, not your actual software.

- Few tools publish detection rates against known bugs, so teams cannot tell signal from plausible-sounding noise.

3. Goals and non-goals

Goals (hackathon)

- G1: A working swarm of at least 20 interacting agents running against a real app.
- G2: Demonstrably find emergent, multi-user bugs that a single-persona baseline misses.
- G3: A measured benchmark: recall, precision, cost, repeatability, against two baselines.
- G4: Every reported finding is verified by deterministic replay.
- G5: A compelling 2-minute live demo with a recorded fallback.

Non-goals

- Load or throughput testing. Concurrency here is targeted: a few agents hit the same action at once to expose correctness bugs, not high-volume traffic.
- Testing apps we do not own or have permission to test.
- Production deployment, auth, multi-tenant hosting, mobile apps.
- Generic support for any app. We ship one demo app plus a thin adapter interface.
- Guaranteed auto-fixing (stretch goal only).

4. Users

- **Primary:** early-stage startups launching multi-user products where users affect each other (marketplaces first, then booking, fintech, and community apps). They ship fast, often with AI-generated code, and have no QA or trust-and-safety team. Hackathon focus: one category, a marketplace.
- **Secondary:** trust and safety engineers who want to probe moderation and abuse defenses.
- **Audience for the hackathon:** judges who need to see the gap, the mechanism, and the proof within two minutes.

5. Product concept

Run flow

1. **Configure:** target app (OpenAPI spec plus base URL), seed, population mix, budgets.
2. **Populate:** generate agents from persona specs with distinct goals.

3. **Run:** agents act concurrently in rounds. The scheduler can fire synchronized bursts to provoke races.
4. **Detect:** invariant checks, error capture, and LLM triage over event windows.
5. **Verify:** extract a minimal action sequence and replay it on a fresh database snapshot.
6. **Report:** ranked findings with repro steps and an evidence trail of which agents did what.
7. **Fix and re-run (stretch):** Claude Code proposes a patch, the swarm re-runs to confirm.

Agent roster (24-hour build, 6 roles)

Role	Objective	Targets
Casual buyer	Browse and buy normally	Baseline traffic, single-user bugs
Bargain hunter	Pay the lowest price possible	Coupon stacking (E2)
Impatient clicker	Double submit, retry, go back	Oversell race (E1), idempotency
Fraudster	Get a refund and keep the item	Double refund (E4)
Sock puppet operator	Buy own items from second accounts	Self-dealing (E5)
Troll	Reach a user who blocked them	Block bypass (E7)

Design principle for adversaries: each adversary gets a measurable objective and a score, not a vibes-based instruction like be a troll. Research on LLM social simulation shows aligned models drift toward consensus; hard objectives and tool access to attack primitives keep conflict alive.

6. Functional requirements

P0 (must ship)

- **FR-1 Demo app (Bazaar):** a multi-user marketplace with 8 planted bugs (section 7).
- **FR-2 Agent runtime:** persona spec, objective, tool-calling loop, rolling-summary memory, per-agent step and token budgets.
- **FR-3 Action layer:** API-first tools generated from the OpenAPI spec and driven by a swarmtest.yaml config. UI-driven Playwright agents are cut from the 24-hour build.
- **FR-4 Concurrency:** async agents plus a barrier scheduler that makes N agents perform the same action at the same instant.

- **FR-5 Event log:** append-only record of every action, request, response, status, timestamp, agent id, round, and seed.
- **FR-6 Detectors:** 4 to 5 SQL and Python invariant checks plus 5xx and log capture. LLM triage is post-demo.
- **FR-7 Verifier:** replay the minimal action sequence on a fresh DB snapshot; mark findings verified or unverified. An independent detective agent is post-demo.
- **FR-8 Report:** findings ranked by severity with repro steps, evidence trail, and in benchmark mode a mapping to planted bug ids.
- **FR-9 Benchmark harness:** run 3 seeds across two conditions (isolated single-persona vs. swarm) and compute metrics.
- **FR-10 Live dashboard:** simple live agent activity feed, findings stream, scoreboard.

P1 (should ship)

- FR-11 Reproducible runs from a seed.
- FR-12 Live cost and token meter with a hard budget cap and kill switch.
- FR-13 Persona definitions in YAML so users can add their own.
- FR-14 Side-by-side comparison view of baseline versus swarm results.

P2 (stretch)

- FR-15 Fix loop using Claude Code in headless mode.
- FR-16 GitHub Action that posts a survival summary on a PR.
- FR-17 Adapter for a second app to show generality.
- FR-18 Adaptive adversaries that remember what was blocked across runs.
- FR-19 Chaos injection (dependency timeouts) during runs.

7. Demo app and planted bugs

Bazaar: users register, list items, search, add to cart, check out, message sellers, review items, request refunds and disputes, block users, and report listings. Each bug has a hidden ground-truth id and an oracle used only for scoring. Bug list is frozen before any benchmark run.

Single-user bugs (baseline should find most of these)

ID	Description
S1	Checkout returns 500 on a specific empty-cart edge case
S2	Negative quantity accepted in cart

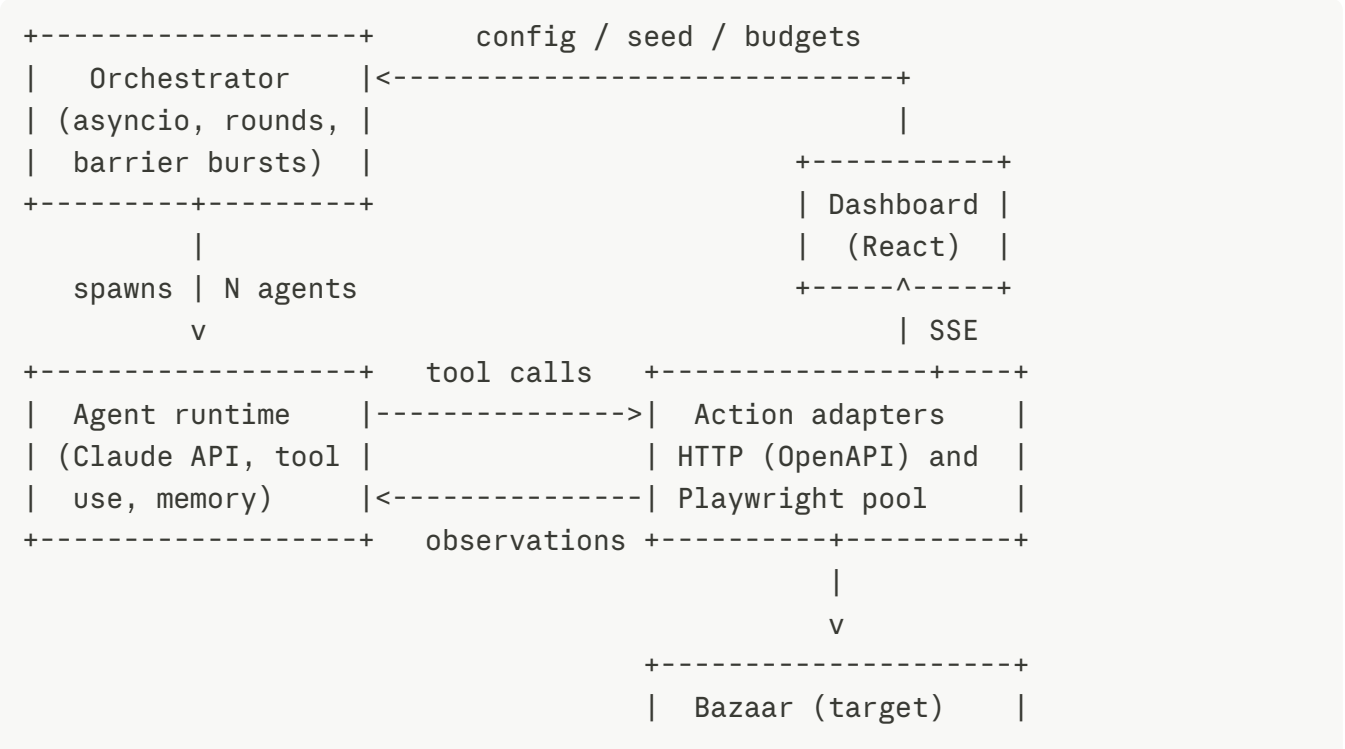
ID	Description
S4	Search crashes on a special character

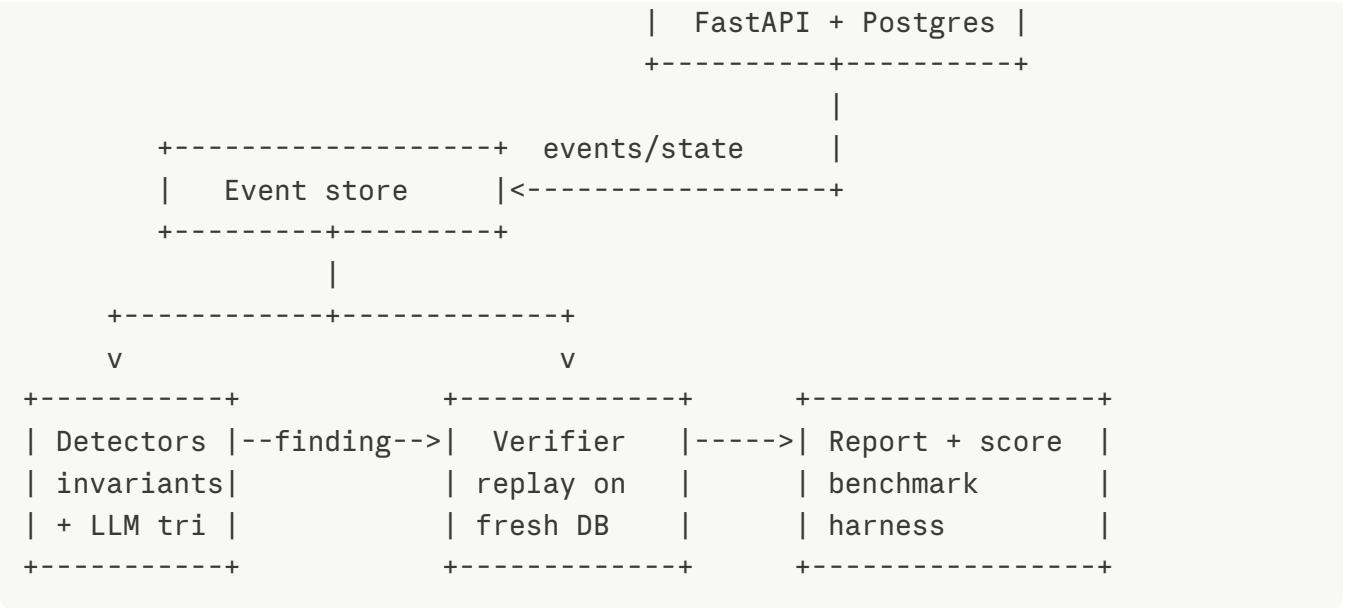
Emergent bugs (need multiple interacting users)

ID	Description
E1	Oversell: two buyers check out the last unit at the same instant
E2	Coupon stacking via concurrent apply requests
E4	Refund plus dispute credit pays out twice
E5	Self-dealing: no same-owner check, so sock puppets inflate rank and rating
E7	Block bypass: blocked user can still reach the person through item question threads

Cut from the 24-hour build and kept as backlog: S3 (stored script injection), E3 (price change between cart and checkout), E6 (review ring), E8 (brigading via false reports). Implement every bug behind a feature flag so Bazaar v2 (fixes applied) is one config switch, which Phase 2 needs.

8. Architecture





Adapter interface (so other apps can plug in later): an OpenAPI spec, an auth and seed-data fixture, and an invariants file. Nothing else.

9. Tech stack

Layer	Choice	Why
Swarm runtime	Python 3.11, asyncio, httpx, Anthropic SDK	Fast to build, good async story, first-class tool use
Bulk agents	claude-haiku-5-5	Cheap and fast enough for 20+ concurrent agents
Triage, detective, report	claude-sonnet-5-5	Stronger reasoning where quality matters
UI-driven agents	Playwright (Chromium); post-demo, not in the 24-hour build	Realism for a small subset; optional browser-use
Demo app backend	FastAPI, SQLAlchemy, Postgres 16	Real concurrency so race bugs are genuine
Demo app frontend	React plus Vite, minimal styling	Needed for UI agents and the demo
Event store	Postgres separate schema (or SQLite file) plus JSONL export	Queryable and replayable
Detectors	SQL assertions plus Python checks plus LLM triage	Oracles first, LLM second

Layer	Choice	Why
Dashboard	React, Tailwind, SSE, react-force-graph or d3	Live visuals sell the demo
Packaging	Docker Compose, Makefile	One command to run everything
Fix loop (stretch)	Claude Code CLI, headless	Closes the find-fix-confirm loop
Evaluation	pytest, pandas, matplotlib	Metrics and charts for the pitch

Confirm current model names, rate limits, and pricing in the Anthropic docs before the event, and pre-check your account limits for 20+ concurrent agents.

10. Key design decisions

1. **API-first for scale, UI for realism.** Most agents call the app API through generated tools; a few drive a real browser.
2. **Objective-driven adversaries.** Each has a score and attack primitives, which counters consensus drift.
3. **Deterministic verification.** LLM nondeterminism is removed at verification time by replaying the logged action sequence on a fresh snapshot.
4. **Oracles before vibes.** Invariants (inventory never negative, money conserved, no duplicate refunds, one review per verified purchase) are primary. LLM triage is secondary. Nothing is reported unverified without a clear label.
5. **Cost control.** Haiku for bulk, step and token caps per agent, rolling memory summaries, prompt caching, start at 20 agents.
6. **Isolation.** Every run starts from a seeded snapshot of the database.

11. Evaluation plan

Conditions

- **A:** scripted end-to-end suite. Cut from the 24-hour build.
- **B:** single-persona isolated agents, same model and same token budget, no inter-agent interaction.
- **C:** Swarmtest full swarm.
- **D (optional):** an external open-source persona tester run on the same app.

Metrics

- Recall, overall and split into single-user versus emergent bugs.

- Precision of reported findings, before and after the verifier.
- Time to first find and time to all finds.
- Cost per run (tokens and dollars).
- Repeatability: share of seeds in which each bug is found.

Protocol

- Freeze the planted-bug list before running.
- Tune prompts on one dev seed only; report on three held-out seeds.
- Equal token budget across conditions B and C.
- Match findings to bug ids with an LLM judge plus a manual spot-check.
- Report mean and range, not best run.

Hypotheses to test (do not claim unless measured): emergent-bug recall for C clearly exceeds B; verified precision at or above 80 percent; single-user recall similar across B and C.

12. Demo script (2 minutes)

1. Run the baseline (isolated single-persona agents) on Bazaar. Show mostly green, a few single-user finds.
2. Launch the swarm. Show the live activity graph with normal users, then adversaries.
3. Watch one emergent bug surface live (for example oversell or refund abuse).
4. Show the verifier replaying it on a fresh database, marked verified.
5. Show the scoreboard: planted bugs found, baseline versus swarm, false positives, cost.
6. Stretch: Claude Code patches one bug, the swarm re-runs, the bug is gone.

Always keep a pre-recorded seeded run as fallback.

13. Plan, roles, and timeline (24 hours, 4 people)

Roles

- **Dev 1, Bazaar and demo:** Bazaar app, 8 planted bugs behind feature flags, OpenAPI spec, seed data, invariants.sql, and a reset-db snapshot (hours 0 to 8). Then the live dashboard, the Bazaar v2 switch, and the demo script and recording. Owns apps/bazaar and dashboard.
- **Dev 2, Swarm runtime:** CLI and swarmtest.yaml, persona specs, Claude tool-use agent loop, tools generated from the OpenAPI spec, async orchestrator, barrier bursts, budgets and kill switch, seeds, and an isolated mode for the baseline. Phase 2: per-agent memory and the Meko adapter. Owns swarm and adapters.

- **Dev 3, Detection, verification, and benchmark:** event store library, invariant runner, error capture, finding builder, replay verifier, report, benchmark harness and charts. Phase 2: MemoryStore and Run 2 classification (fixed, still failing, regressed). Owns detect, verify, bench, and memory.

Handoffs

- Hour 3: Dev 1 gives Dev 2 the Bazaar OpenAPI spec (a stub is fine, then the real one).
- Hour 5: Dev 2 has events flowing into Dev 3's event store.
- Hour 6: Dev 1 gives Dev 3 invariants.sql and the reset-db command.
- Hour 10: Dev 3 gives Dev 1 the finding JSON for the dashboard.
- Bug build order for Dev 1: E1 and E4 first (easiest to detect), then E2, E5, E7, then S1, S2, S4. The hour-8 checkpoint needs E1 working by about hour 5.
- Dev 2 and Dev 3 pair on the hour-8 checkpoint. If Dev 3 is overloaded, Dev 1 takes the report generator after Bazaar.

Timeline

Hours	Milestone
0 to 2	Repo, Docker Compose, OpenAPI contract, event and finding schemas agreed
2 to 8	Bazaar v1 with 8 bugs behind feature flags; agent loop with 3 roles on API tools; event log plus 3 invariants; dashboard skeleton
8	**Checkpoint:** swarm finds its first emergent bug end to end
8 to 14	All 6 roles; barrier bursts; verifier replay; live feed
14 to 16	Benchmark runs (2 conditions, 3 seeds)
16	**Core gate:** at least 3 verified emergent bugs and one benchmark result. Phase 2 starts only if this passes
16 to 21	Phase 2: memory layer and Meko integration (section 17), in priority order; otherwise polish and more seeds
21 to 24	Demo rehearsal, recorded fallback, submission buffer

Cut list (drop in this order if behind): Meko integration (skipped entirely if the hour-16 core gate fails), fix loop, GitHub Action, then reduce the roster to 4 roles. Already cut from the 24-hour build: UI agents, detective agent, LLM triage, scripted end-to-end baseline.

14. Risks and mitigations

Risk	Mitigation
Agents are too agreeable and produce no real conflict	Objective-driven adversaries with scores and scripted attack primitives
Cost or rate limits block concurrency	Haiku for bulk, step caps, caching, fewer agents, pre-recorded demo run
Race conditions not triggered reliably	Barrier scheduler, database-level concurrency, repeated bursts
Findings sound plausible but are false	Invariant oracles plus replay gate; report unverified findings separately
Benchmark looks rigged	Frozen bug list, held-out seeds, publish scripts and bug list
Judges say it is just an existing persona tester	Show emergent bugs that condition B cannot find, with numbers
Live demo fails	Seeded run, recorded fallback, local-only setup
Scope creep	Cut list in section 13; checkpoints at hours 10 and 18

15. Safety and ethics

- Test only apps you own or have explicit permission to test. Demo runs fully local.
- Use synthetic data only; no real personal information.
- Adversarial content is simulated with benign placeholders: the filter-evasion test uses an innocuous banned-word list, not real slurs.
- Hard budget caps, rate limits, and a global kill switch.
- Follow Anthropic usage policies; do not point the swarm at third-party systems.

16. Distribution and onboarding

Customers run Swarmtest from the command line against a staging copy of their app, driven by one config file. No SDK or library import is needed because Swarmtest tests from the outside, like real users.

Hackathon build: CLI plus swarmtest.yaml

```
app:
  base_url: http://localhost:8000
  openapi: ./openapi.json
accounts:
  seed_script: ./seed_test_users.py
invariants: ./invariants.sql
population:
  casual_buyer: 6
  bargain_hunter: 4
  impatient_clicker: 4
  fraudster: 2
  sockpuppet_operator: 2
  troll: 2
budget:
  max_dollars: 5
  seed: 1
```

Run it with `swarmtest run --config swarmtest.yaml`. Customers bring their own Anthropic API key, so there is no billing system to build. Swarmtest needs a disposable staging environment it can reset for replay; it must never run against production.

Roadmap (slides only, not built)

- GitHub Action that runs the swarm on pull requests and before launch.
- Hosted service: account and API key, uploaded spec, staging URL connection, run history, team dashboard, usage-based billing.

17. Phase 2: memory with Meko (after the core works)

Phase 2 starts only after the hour-16 core gate passes, and nothing in Phase 1 depends on it. Meko is Yugabyte's agent-native data infrastructure: a project's memory, knowledge, conversations, and decision traces live in a portable Datapack that agents reach over MCP (launch coverage, AWS Marketplace listing). Its API details are unverified here: check the hackathon docs for endpoint, auth, available actions, rate limits, and local setup before building.

The idea: Run 1 finds bugs and saves each as a verified, replayable check. The company fixes them. Run 2 recalls the saved checks, re-runs them first, and reports each as fixed, still failing, or regressed, then spends the remaining budget exploring for new bugs.

Swarmtest need	Meko feature	Priority
Save each verified finding and its replay sequence	Knowledge in a project Datapack	Must
Run 2 re-checks saved findings: fixed, still failing, or regressed	Read knowledge over MCP	Must
Personas remember what worked or got blocked	Per-agent memory	Should
Why an agent acted and why a finding was flagged	Decision traces	Should
Agents share discoveries mid-run	Collective memory	Stretch
Developer's coding agent reads findings and fixes them	MCP, same Datapack	Stretch

Integration rules

- Build a MemoryStore interface from the start (save_finding, recall_findings, save_agent_memory, recall_agent_memory, record_trace) with a local implementation first (Postgres or JSON files). Phase 1 and the demo run on it.
- The Meko adapter is a second implementation of the same interface, built in Phase 2.
- The event log and replay data stay in our own Postgres; Meko holds memory, knowledge, and traces.
- If the Meko adapter slips, the demo runs on the local implementation and the slide says the memory layer is pluggable.

Phase 2 demo extension

1. Run 1 on Bazaar v1 finds bugs and saves them.
2. Ship Bazaar v2 with one config switch: two bugs fixed, one still open, and backlog bug E3 enabled as a new regression.
3. Run 2 recalls memory, confirms the two fixes, flags the open bug, catches the regression, then explores for more.

Phase 2 metrics: tokens and time to first find in Run 2 with versus without memory; share of earlier findings classified correctly; LLM calls needed for regression re-checks (target zero, since replays are deterministic).

18. Business model and scaling

Exploration is expensive and regression is cheap. The swarm uses LLMs to discover an exploit once; every verified exploit becomes a deterministic replay that re-runs on later releases with no LLM calls, so cost per customer grows more slowly than usage.

Go-to-market stages

1. **Wedge:** open-source CLI plus config file for early-stage startups launching marketplaces, booking, or fintech products. Customers bring their own API key. This is the demo.
2. **Distribution:** a GitHub Action that runs the swarm on pull requests and before launch.
3. **Expansion by vertical:** persona-and-invariant packs per app category (marketplace, community, fintech, collaboration).
4. **Hosted product:** managed staging, run history, regression tracking, team dashboards, an API key, and usage-based pricing.
5. **Later:** enterprise and trust-and-safety features such as compliance reports, custom adversary libraries, and private deployment.

Business model: open-source core for adoption, with paid hosted tiers priced on usage (runs or simulated agent-hours) plus a team plan for history and CI. Pricing placeholder: [$X per run], to be set from the measured cost per run.

Moat: per-app memory of how an app breaks (Phase 2), a published benchmark of verified findings, vertical packs, and CI integration. Opt-in sharing of anonymized attack patterns across customers is a roadmap idea, not a demo claim.

Numbers to show judges: cost per run and cost per verified bug from our own benchmark. No market-size claims without a source.

Honest risk: a larger testing vendor could add multi-user simulation. Our answer is verified findings, a published benchmark, vertical packs, and being embedded in CI.

19. Open questions

- Actual team size, time limit, and any sponsor or track requirements?
- Judging criteria: weight on technical depth, novelty, or business value?
- Preferred language split (Python versus TypeScript)?
- API credit budget per run, and rate limits on the account?
- Is the second-app adapter (FR-17) worth the time to show generality?

Appendix A: persona spec example

```
id: fraudster-01
role: fraudster
model: claude-haiku-5-5
objective: Receive a refund while keeping the item
score: refunds_received minus items_returned
constraints:
  max_steps: 40
  max_tokens: 60000
tools: [search, add_to_cart, checkout, open_dispute, request_refund, message]
style: patient, avoids obvious patterns, tries a different route when blocked
memory: rolling summary every 8 steps
```

Appendix B: event and finding fields

Event: run_id, seed, round, timestamp, agent_id, role, tool, request, response_status, response_body_hash, db_changes, latency_ms, tokens_used.

Finding: id, title, severity, type (single-user or emergent), evidence_events, involved_agents, minimal_repro_sequence, oracle_triggered, verified (yes or no), mapped_bug_id (benchmark mode only), suggested_fix.

Appendix C: agent loop (pseudocode)

```
for each round:
    for each agent (concurrently):
        observe state and recent events
        choose a tool call toward the objective
        execute through the adapter
        log the event
        update rolling memory
    scheduler may insert a barrier: N agents run the same tool at the same
instant
    detectors scan the new event window
```

Appendix D: repository layout

```
swarmtest/
  apps/bazaar/        # target app: api, web, seed data, planted bugs
  swarm/              # orchestrator, agent runtime, personas, scheduler
  adapters/           # http tools from OpenAPI, playwright pool
  detect/             # invariants, log capture, llm triage
  verify/             # replay, detective agent
  bench/              # conditions A, B, C, metrics, charts
  dashboard/          # react app, sse stream
  docker-compose.yml
  Makefile
  README.md
```